

HTML

The Language of the Web

HyperText Markup Language • MERN Stack Series

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<body>
```

```
</body>
```

```
</html>
```

What is HTML?



HTML = HyperText Markup Language

The standard language for creating web pages — invented by Tim Berners-Lee in 1991.



It is the skeleton of every webpage

Defines the structure and meaning of content — not how it looks.



Every website you've visited is built on HTML

Right-click any page → View Page Source → That's raw HTML.



HTML uses tags to mark up content

`<h1>This is a heading</h1>` — browser reads tags and knows what to display.



Works alongside CSS and JavaScript

HTML = structure. CSS = style. JS = behaviour. They work as a team.



You NEED HTML before React

React generates HTML. If you don't know HTML, you can't debug React output.

HTML, CSS & JavaScript — The Web Trio



HTML

Structure & Content

What is on the page

Headings, paragraphs, images, links, forms, tables — the raw content of the page.

```
<html>
```



CSS

Style & Layout

How it looks

Colors, fonts, spacing, flexbox layouts, animations, responsive design.

```
.style {}
```



JS

Behaviour & Logic

What it does

Click events, form validation, fetch data from server, update the DOM.

```
function()
```

The HTML Boilerplate

The starting template every HTML file needs.

HTML Boilerplate — Your Starting Template

```
<!DOCTYPE html>
<html lang="en">

  <head>
    <meta charset="UTF-8" />
    <meta name="viewport"
      content="width=device-width,
        initial-scale=1.0" />
    <title>My First Page</title>
    <link rel="stylesheet"
      href="style.css" />
  </head>

  <body>

    <h1>Hello, World!</h1>

    <script src="app.js"></script>
  </body>
</html>
```

<!DOCTYPE html>

Activates HTML5 mode in the browser.

<html lang="en">

Root element. lang= helps SEO & screen readers.

<head>

Invisible page settings (not shown to user).

charset="UTF-8"

Supports Hindi, emoji, all world characters.

viewport meta

Makes page mobile-friendly. Always include!

<title>

Text shown in browser tab + Google results.

<body>

Everything visible goes here.

Core Concepts

Tags, elements, attributes — the building blocks.

</> { } []

Anatomy of an HTML Element

```
<p class="intro">Hello, World!</p>
```



Opening Tag

`<tagname>` — tells browser where element starts



Closing Tag

`</tagname>` — the `/` means closing. Required for most tags.



Self-Closing

`<br />` `<img />` `<input />` `<meta />` — no content, no closing tag



Attributes

`key="value"` pairs inside the opening tag. Add extra info.

Most-Used HTML Attributes

#

id="name"

Unique on page. Used in CSS as #name and JS as getElementById()

.

class="name"

Reusable label. Multiple elements can share. CSS: .name

**href="url"**

Used in <a> tags. Defines the link destination URL.

**src="path"**

Used in <script>. Points to an external file.

**alt="text"**

Image description for screen readers & SEO. Always add it!

**style="css"**

Inline CSS. Works but avoid in big projects — use stylesheets.

Headings & Text Tags

Structure your content with meaning.

Headings — h1 to h6

```
<!-- 6 heading levels in HTML -->
<h1>Main Page Title</h1>
<h2>Section Title</h2>
<h3>Sub-section</h3>
<h4>Sub-sub-section</h4>
<h5>Rarely used</h5>
<h6>Smallest heading</h6>

<!-- Real-world blog structure -->
<h1>My Developer Blog</h1>
  <h2>JavaScript Tips</h2>
    <h3>Arrow Functions</h3>
    <h3>Destructuring</h3>
  <h2>HTML Basics</h2>
    <h3>Boilerplate</h3>
```

Only ONE <h1> per page

Like the main headline of a newspaper. Google reads it first.

Follow the hierarchy

Never skip levels — h1 → h2 → h3. Don't jump h1 to h4.

Headings = SEO

Search engines use headings to understand your page topic.

Don't pick headings for size

Use CSS to change size. Use headings for MEANING.



Think of it as a book:

h1 = Book title, h2 = Chapter, h3 = Section title

Text Formatting Tags

```
<!-- Paragraph (most common text tag) -->
<p>This is a paragraph of text.</p>
```

```
<!-- Inline text formatting -->
<p>
  I am <strong>bold & important</strong>.
  I am <em>italicised for emphasis</em>.
  I am <mark>highlighted</mark>.
  I am <del>strikethrough</del>.
  H<sub>2</sub>O and x<sup>2</sup>.
</p>
```

```
<!-- Line break & horizontal rule -->
<p>Line one.<br />Line two.</p>
<hr />
```

```
<!-- Inline code display -->
<p>Use <code>console.log()</code>
to debug your code.</p>
```

```
<!-- Block quote -->
<blockquote>
  "Any fool can write code that a
  computer can understand." — Fowler
</blockquote>
```

<p> — Paragraph

Block element. Always starts on a new line. Main way to write text.

 vs

 = semantically important. = just bold visually. Prefer .

 vs <i>

 = stressed emphasis. <i> = visually italic. Prefer .

 — Line break

Just moves to next line. No spacing. Use sparingly — use <p> for new paragraphs.

Semantic tags matter!

Screen readers announce differently. Good accessibility = good UX.

Links & Images

Making pages navigable and visual.

Links — The <a> Anchor Tag

```
<!-- External link -->
<a href="https://google.com">
  Go to Google
</a>

<!-- New tab (+ security attr) -->
<a href="https://github.com"
  target="_blank"
  rel="noopener noreferrer">
  My GitHub
</a>

<!-- Internal page -->
<a href="about.html">About Us</a>

<!-- Jump to section (hash link) -->
<a href="#contact">Go to Contact</a>
<section id="contact">...</section>

<!-- Email & download -->
<a href="mailto:hi@gmail.com">
  Email Me
</a>
<a href="cv.pdf" download>
  Download CV
</a>
```

href — HyperText Reference

The destination. Without it the link does nothing.

target="_blank"

Opens in new tab. Always pair with rel="noopener noreferrer" for security.

Absolute vs Relative URL

https://google.com = absolute (works anywhere).
/about.html = relative (relative to your site).

Hash links #id

Scrolls to element with that id on the same page. Used in single-page sites.



In React:

You'll use <Link> from react-router — same concept, same attributes.

Images — The Tag

```
<!-- Basic image -->


<!-- Set dimensions -->


<!-- Image from URL -->


<!-- Image as a link -->
<a href="https://google.com">
  
</a>

<!-- Lazy loading for performance -->

```

 is self-closing!

No needed. Always end with />.

alt is REQUIRED

Shown if image fails to load. Read by screen readers. Indexed by Google for SEO.

src path types:

"cat.jpg" same folder • "imgs/cat.jpg" subfolder •
"./cat.jpg" parent folder

Set width + height

Prevents layout shift while page loads. Big performance improvement.

loading="lazy"

Image only loads when user scrolls near it. Huge performance boost on image-heavy pages.

Lists

Organising information cleanly.

</> { } []

Lists — ul, ol, dl

```
<!-- Unordered list (bullets) -->
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>

<!-- Ordered list (numbered) -->
<ol>
  <li>Open VS Code</li>
  <li>Create index.html</li>
  <li>Write HTML</li>
  <li>Open in browser</li>
</ol>

<!-- Nested list -->
<ul>
  <li>Frontend
    <ul>
      <li>HTML → CSS → JS</li>
    </ul>
  </li>
  <li>Backend → Node.js</li>
</ul>

<!-- Description list (glossary) -->
<dl>
  <dt>HTML</dt>
  <dd>Structure of the web</dd>
```

 — Unordered List

Bullet points. Use when ORDER doesn't matter. Shopping list, features, nav links.

 — Ordered List

Numbered 1,2,3... Use when ORDER matters. Steps, rankings, recipes.

 — List Item

Used inside both ul and ol. Can contain any HTML inside it!

<dl> — Description List

<dt> = term, <dd> = definition. Good for glossaries, FAQs, key-value data.



Real-world use:

Navigation menus are ALWAYS built as <a> — styled with CSS to look like a navbar.

Tables

Displaying data in rows and columns.

Tables — Displaying Tabular Data

```
<table>
  <thead>
    <tr>
      <th>Name</th>
      <th>Role</th>
      <th>Salary</th>
    </tr>
  </thead>

  <tbody>
    <tr>
      <td>Rahul</td>
      <td>Frontend Dev</td>
      <td>12 LPA</td>
    </tr>
    <tr>
      <td>Priya</td>
      <td>Backend Dev</td>
      <td>15 LPA</td>
    </tr>
  </tbody>

  <tfoot>
    <tr>
      <td colspan="2">Average</td>
      <td>13.5 LPA</td>
    </tr>
  </tfoot>
</table>
```

Table structure:

`<table>` wraps everything • `<thead>` = header row •
`<tbody>` = data • `<tfoot>` = summary/footer

`<tr>` — Table Row

One horizontal row of data.

`<th>` — Header Cell

Bold + centered by default. Use in `<thead>`.

`<td>` — Data Cell

Regular data cell. Use in `<tbody>` and `<tfoot>`.

colspan & rowspan:

`colspan="2"` → cell spans 2 columns. `rowspan="3"` → spans 3 rows.

⚠ Tables are for DATA only!

Never use tables for page layout. Use CSS Flexbox or Grid for that.

 Quick Check!

Which heading tag should be used for the MAIN title of a webpage?

A <h2> — h1 is reserved for sub-sections

B <title> — it's literally called 'title'

C <h1> — one per page, most important heading

D <header> — because it sits at the top

Discuss with your neighbour before answering! 

Forms & Inputs

Collecting data — the heart of every web app.

Forms — Collecting User Input

```
<form action="/register"
      method="POST">

  <label for="name">Full Name:</label>
  <input type="text"
        id="name"
        name="name"
        placeholder="e.g. Rahul"
        required />

  <label for="email">Email:</label>
  <input type="email"
        id="email"
        name="email"
        required />

  <input type="password"
        name="password"
        placeholder="Min 8 chars" />

  <input type="number"
        name="age"
        min="18" max="60" />

  <button type="submit">
    Register Now
  </button>

</form>
```

<form> attributes:

action= where to send data • method= GET or POST
GET: data in URL (search) • POST: data hidden (login)

<label for=id>

Clicking the label focuses the input. Crucial for accessibility!
for= must match input's id=.

input types:

text email password number tel url date checkbox radio
file hidden range color

name= attribute

The KEY sent to the server. Without name=, the data won't be included in the request!

required / disabled

required = HTML validation before submit. disabled = user can't interact with field.

More Form Elements — Select, Textarea, Checkbox

```
<!-- Multi-line text -->
<textarea name="bio"
  rows="4"
  placeholder="About you...">
</textarea>

<!-- Dropdown -->
<select name="city">
  <option value="">-- Choose --</option>
  <option value="indore">Indore</option>
  <option value="mumbai">Mumbai</option>
</select>

<!-- Checkbox (multiple ok) -->
<input type="checkbox"
  id="terms" name="terms" />
<label for="terms">I agree</label>

<!-- Radio (one choice only) -->
<input type="radio"
  name="gender" value="M" />
<label>Male</label>
<input type="radio"
  name="gender" value="F" />
<label>Female</label>

<!-- File upload -->
<input type="file" accept="image/*" />
```

<textarea>

Multi-line text. rows/cols sets initial size. Resize with CSS. Value goes between the tags.

<select> + <option>

Dropdown menu. value= is what gets sent. Text between tags = what user sees.

Checkbox

Multiple can be checked . Each needs own name=. Value sent only if checked.

Radio buttons

Same name= groups them together. Only ONE can be selected at a time.

<button> types:

type="submit" → sends form • type="reset" → clears • type="button" → does nothing (use with JS)



Quick Check!

In a form, what does the name= attribute on an input do?

A

Displays a label to the user above the input

B

Gives the input a unique CSS id for styling

C

Identifies the data key sent to the server when form submits

D

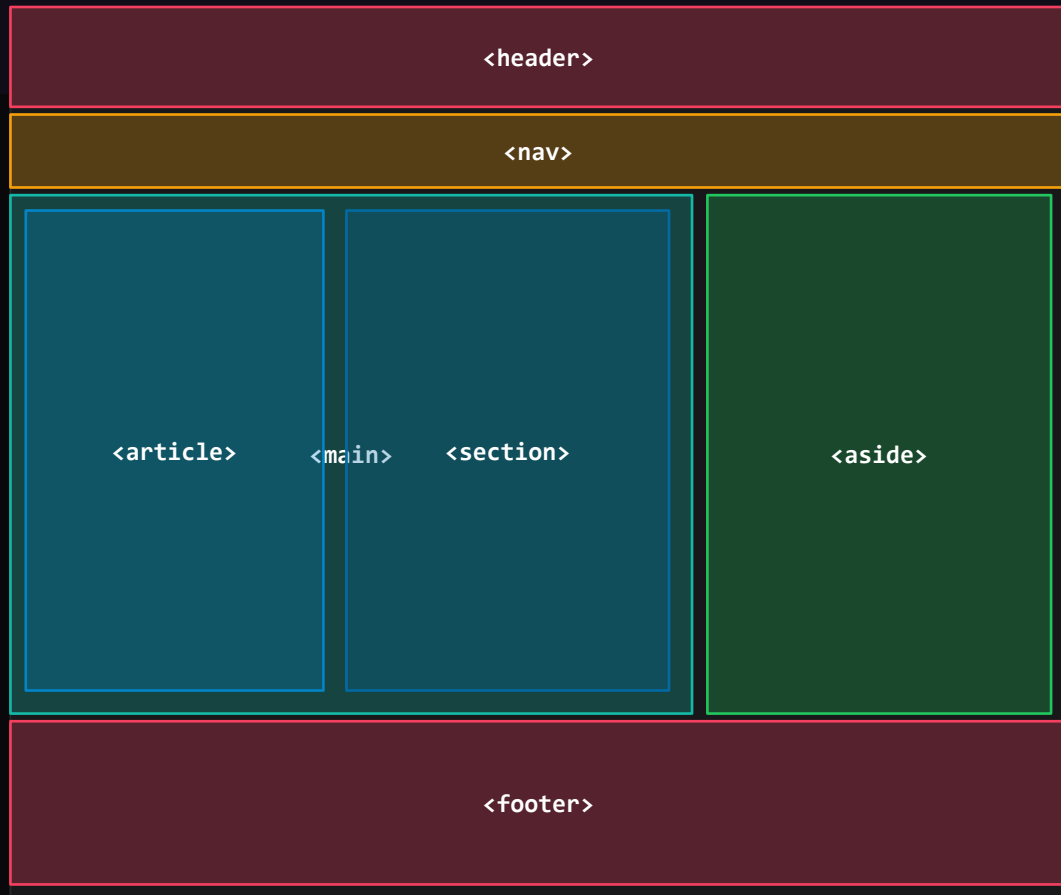
Sets a default placeholder text inside the input

Think about how form data travels to a Node.js backend... 🤔

Semantic HTML

Writing HTML that means something.

Semantic HTML5 — Page Structure Visual



<header>

Logo, site title, hero. Top of page.

<nav>

Navigation menu links.

<main>

Primary content. ONE per page!

<article>

Self-contained: blog post, news.

<section>

Thematic grouping of content.

<aside>

Sidebar, related links, ads.

<footer>

Copyright, links, contact.

Why Semantic HTML Matters



Accessibility

Screen readers understand `<nav>`, `<main>`, `<article>`. A `<div>` has no meaning to them.



SEO

Google understands structure from semantic tags. `<article>` content ranks better.



Readable Code

`<header>` is self-explanatory. 10 nested `<div>`s is a nightmare to maintain.



Browser Features

Reader modes, skip links, and accessibility tools all rely on semantic structure.



The `<div>` trap



```
<div id="nav"><div id="menu">
```



```
<nav><ul><li><a href> — use the right tag!
```



Rule of thumb

Ask: 'Is there a semantic tag for this?' Use `<div>` only when NOTHING else fits.

Media Elements

Video, audio, and embedded content.

Video, Audio & iFrames

```
<!-- HTML5 Video -->
<video controls
  autoplay
  muted
  loop
  width="640">
  <source src="intro.mp4"
    type="video/mp4" />
  <source src="intro.ogg"
    type="video/ogg" />
  Browser doesn't support video.
</video>

<!-- HTML5 Audio -->
<audio controls>
  <source src="song.mp3"
    type="audio/mpeg" />
</audio>

<!-- YouTube embed (iFrame) -->
<iframe
  src="https://youtube.com/embed/XYZ"
  width="560" height="315"
  allowfullscreen>
</iframe>
```

<video> attributes:

controls — show play bar • autoplay — start on load • muted — required for autoplay! • loop — repeat

<source> tags:

Browser picks first format it supports. Always include mp4 — has best browser support.

Fallback text:

Content between <video> tags shows only if browser can't play video.

<audio>:

Same pattern as video. controls attribute is important — gives user a play/pause bar.

<iframe> — embed anything:

YouTube, Google Maps, payment widgets. Use allowfullscreen for video. sandbox= adds security.

The <head> Section

Invisible settings that power your page.

Meta Tags & Head Elements

```
<head>
  <meta charset="UTF-8" />

  <meta name="viewport"
    content="width=device-width,
      initial-scale=1.0" />

  <!-- Google search description -->
  <meta name="description"
    content="Learn HTML in this
      MERN stack course." />

  <!-- Social media share preview -->
  <meta property="og:title"
    content="My Awesome Site" />
  <meta property="og:image"
    content="preview.jpg" />
  <meta property="og:description"
    content="Best MERN course" />

  <!-- Browser tab icon -->
  <link rel="icon"
    href="favicon.ico" />

  <title>My Page — 60 chars max</title>
</head>
```

charset UTF-8

Without this, ₹ symbols, Hindi text, emojis may display as garbage.

viewport meta

Without this, mobile users see a tiny zoomed-out desktop page. Always include!

description meta

Shown under your link in Google search results. Keep it 150-160 characters.

Open Graph (og:)

Controls how your link looks when shared on WhatsApp, LinkedIn, Twitter, Facebook.

favicon

The tiny icon in the browser tab. People notice when it's missing!

<title>

Browser tab text + used by Google. Keep under 60 characters.

Essential Extras

div, span, buttons, comments & entities.

</> { } []

div, span, Comments & HTML Entities

```
<!-- BLOCK container for layout -->
<div class="card">
  <h2>Card Title</h2>
  <p>Card content here.</p>
</div>

<!-- INLINE container for partial text -->
<p>My name is
  <span class="highlight">Rahul</span>
  and I love coding.
</p>

<!-- Comments – hidden from users -->
<!-- Single line comment -->
<!--
  Multi-line comment
  Great for notes in your code
-->

<!-- HTML Special Characters -->
<p>&lt;div&gt; looks like this</p>
<p>Copyright &copy; 2025</p>
<p>Price: &#8377;999</p>
<p>Non-breaking&nbsp;space here</p>
```

<div> — Block Container

Full width, starts new line. Used for grouping content for layout. Most-used tag in React apps.

 — Inline Container

Only as wide as content. Stays within text flow. Use to style a part of a sentence.

Block vs Inline:

Block: div p h1-h6 ul table — always new line.
Inline: span a strong em img — stays in text.

HTML Entities:

< → < > → > & → & © → © → space ₹ → ₹



Ctrl+/ in VS Code

Toggles <!-- --> comment on any selected line. Huge time-saver!

Buttons, Script & Style Tags

```
<!-- Button types -->
<button type="button"
  onclick="alert('Hi!')">
  Click Me
</button>

<button type="submit">Submit</button>
<button type="reset">Clear</button>
<button disabled>Not Clickable</button>

<!-- External JS (end of body) -->
<script src="app.js"></script>

<!-- Or with defer in <head> -->
<script src="app.js" defer></script>

<!-- External CSS (in <head>) -->
<link rel="stylesheet"
  href="style.css" />

<!-- Inline CSS (testing only) -->
<style>
  body { background: #09090B; }
  h1 { color: #14B8A6; }
</style>
```

button type=

"submit" → sends form • "reset" → clears form •
"button" → nothing (for JS). Default is "submit"!

disabled attribute

Greyed out, can't click. Used for validation — disable submit until all fields are filled.

Script placement:

End of <body> is safest — HTML loads first then JS runs. Or use defer in <head>.

defer vs async:

defer → runs after HTML is parsed (most scripts). async → runs as soon as downloaded (analytics).

<link> vs <style>:

<link href=file> → external CSS (use for real projects).
<style> → CSS inside HTML (testing only).

 Quick Check!

What is the key difference between `<div>` and `<span>`?

- A `<div>` is for CSS only; `<span>` is for JavaScript interactions
- B `<div>` is a block element (new line, full width); `<span>` is inline (stays in text)
- C They are interchangeable — just different naming conventions
- D `<span>` is the newer replacement for `<div>` in HTML5

Remember: Block vs Inline is one of the MOST important CSS concepts too! 

HTML Best Practices — Write Like a Pro



Always add alt to images

alt="" for decorative images. Descriptive text for meaningful ones.



Label every form input

<label for="x"> paired with id="x" on the input. No label = bad accessibility.



Use semantic tags first

Ask: 'Is there a tag for this?' Use <nav>, <article>, <section> before reaching for <div>.



Validate your HTML

validator.w3.org — the official W3C HTML validator. Catches hidden bugs.



Indent nested elements

Children indented 2 spaces inside parent. VS Code auto-formats on save with Prettier.



Close all your tags

Forgetting </div> or causes bizarre layout bugs. Use a linter!

VS Code Power Tips for HTML



! + Tab

Generates the full HTML boilerplate instantly. Use every time you create a new file.



Emmet Abbreviations

`ul>li*3` → ul with 3 lis. `div.card>h2+p` → div.card containing h2 and p.



Live Server Extension

Auto-refreshes browser on save. No more manual F5! Install from Extensions panel.



Prettier Extension

Auto-formats HTML on every save. Keeps indentation and style perfectly consistent.



Ctrl+/ to Comment

Toggle `<!-- -->` comment on selected lines. Works in HTML, CSS, and JS.



Ctrl+P — Quick Open

Open any file in your project instantly by typing its name. Faster than the file tree.

HTML Quick Reference — Tags You'll Use Daily

Structure	Text	Links & Media	Lists
<code><!DOCTYPE html></code> <code><html></code> <code><head></code> <code><body></code> <code><title></code> <code><meta></code> <code><link></code> <code><script></code>	<code><h1>–<h6></code> <code><p></code> <code></code> <code></code> <code></code> <code>
</code> <code><hr /></code> <code><blockquote></code>	<code><a href></code> <code></code> <code><video></code> <code><audio></code> <code><iframe></code>	<code></code> <code></code> <code></code> <code><dl></code> <code><dt></code> <code><dd></code>
Tables	Forms	Semantic	Containers
<code><table></code> <code><thead></code> <code><tbody></code> <code><tr></code> <code><th></code> <code><td></code>	<code><form></code> <code><input></code> <code><label></code> <code><button></code> <code><select></code> <code><textarea></code>	<code><header></code> <code><nav></code> <code><main></code> <code><article></code> <code><section></code> <code><aside></code> <code><footer></code>	<code><div></code> <code></code> <code><!-- comment --></code>



Practice Projects — Build These Tonight!

1

Personal Profile Page

Photo, name (h1), bio (p), skills list (ul), social links (a). Pure structure, no CSS.

2

Blog Page

header + nav, 3 articles with h2/p/img, and a footer. Focus on semantic HTML.

3

Product Landing Page

Hero section, features (div cards), pricing table (HTML table), contact form.

4

Registration Form

Full form: text, email, password, gender (radio), city (select), T&C (checkbox), submit.

5

Mini Portfolio

Combine everything: nav, hero, projects section, about me, contact form, footer.

🎯

Bonus: Recreate a Page

Pick any simple website (Wikipedia article, news page) and recreate its HTML structure.

What's Next — Your MERN Journey

01



HTML

Structure — you're here!

02

CSS

Style, Flexbox, Grid, Animations

03

JavaScript

DOM, Events, Fetch, Async/Await

04

React

Components, Hooks, State, JSX

05

Node + Express

Server, REST APIs, Middleware

06

MongoDB

Database, Mongoose, Full MERN App!

**Keep Building.
Keep Breaking.
Keep Learning.**

The best way to learn HTML is to open VS Code right now and build something.